

# claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

## Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_34671ac1-e8e\agent-add11ddc05638f91a.jsonl`
- SHA-256: `56a6962ff0da97ba2397de2d72a414a7a095116969cce5ea6612e23ece518ead`
- Source modified: `2026-06-21T10:21:08+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_34671ac1-e8e`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`
```

## Transcript

### 1. user (2026-06-21T10:18:25.337Z)

Review the COMPUTE\_DECOUPLED\_SERVING \*\*M5\*\* change just committed on branch feat/serving-m5-truly-local-checks (HEAD). Inspect it with: git diff origin/master...HEAD , and read the full files:

- backend/config/startup.py (NEW ? the per-profile fail/warn-fast checks)
- backend/config/presets.py (added probe\_cuda\_available)
- backend/config/\_\_init\_\_.py (exports)
- backend/main.py (server main(): enforce after global\_config load; sys.exit(1) on fatal)
- backend/worker/\_\_main\_\_.py (\_run\_startup\_checks helper; called in cmd\_infer + cmd\_train after \_apply\_overrides; returns rc 2 on fatal)
- tests/test\_startup\_checks.py (NEW, L1) + tests/test\_profile\_parity.py (NEW, L4)

The contract (must hold): DEFAULT-OFF ? when deployment.profile==" NOTHING changes vs pre-M5 (byte-identical). Only an explicitly-selected profile triggers checks. cloud-serving + non-cloud storage = the ONLY fatal error (everything else is a warning) deliberately, to avoid false-positives (the CUDA probe is best-effort; ADC creds can't be confirmed offline). One isolated PR, default-OFF discipline. Report ONLY real, grounded issues (file:line). If a lens finds nothing real, return an empty findings array ? do not invent nits.

LENS = REPO CONVENTIONS / INTEGRATION / HYGIENE. (1) Logging: do the [STARTUP] lines follow the repo's logging conventions; is logging double-configured or emitted at import? (2) Imports: any circular-import risk (backend.config.\_\_init\_\_ now imports .startup which imports ...)? any unused import? (3) Does the worker probe-cuda call belong inside \_run\_startup\_checks (is it always invoked even when profile==""? trace probe vs the early-return in run\_startup\_checks)? (4) mypy/ruff: any annotation or lint issue the CI lanes would catch (Optional handling, getattr returns Any)? (5) Naming/consistency with M1?M4 (presets.py style, the DeploymentConfig Literal). (6) Anything in main.py's import-block refactor (the multiline from backend.config import) that could break? Report grounded hygiene/integration issues only.

### 2. assistant (2026-06-21T10:18:28.080Z)

I'll review the M5 change. Let me start by pulling the latest state and inspecting the diff.

### 3. user (2026-06-21T10:18:31.577Z)

```
---BRANCH---
feat/serving-m5-truly-local-checks
---HEAD---
6f78acd feat(serving): M5 ? per-profile startup checks + L4 profile-parity test (default-OFF)
fefa412 docs(serving): resolve §8 design picks (D16/D17) + reconcile per-video P3 vs M3 (#285)
06ff4b7 test(api): lock gs://|gcs:// /api/process -> queryable API-DB video_id (B-P1 guard) (#284)
```

### 4. user (2026-06-21T10:18:31.800Z)

```
backend/config/__init__.py
```

```
| 13 +-

```

```

backend/config/presets.py | 12 ++
backend/config/startup.py | 179 ++++++
backend/main.py | 18 +-
backend/worker/__main__.py | 23 ++-
.../runlogs/RUNLOG_truly-local_TEMPLATE.md | 49 +++++
tests/test_profile_parity.py | 139 ++++++
tests/test_startup_checks.py | 181 ++++++
8 files changed, 611 insertions(+), 3 deletions(-)

```

## 5. assistant (2026-06-21T10:18:34.415Z)

Let me read all the changed files and the full diff.

## 6. user (2026-06-21T10:18:35.541Z)

```

diff --git a/backend/config/__init__.py b/backend/config/__init__.py
index 493c535..9b276c0 100644
--- a/backend/config/__init__.py
+++ b/backend/config/__init__.py
@@ -16,7 +16,13 @@ from .models import ( # re-export common ones
    StorageConfig,
    ValidationConfig,
)
-from .presets import PROFILE_PRESETS, suggest_profile
+from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
+from .startup import (
+    StartupCheck,
+    StartupCheckError,
+    enforce_startup_checks,
+    run_startup_checks,
+)

__all__ = [
    "load_config",
@@ -30,4 +36,9 @@ __all__ = [
    "ValidationConfig",
    "PROFILE_PRESETS",
    "suggest_profile",
+    "probe_cuda_available",
+    "StartupCheck",
+    "StartupCheckError",
+    "enforce_startup_checks",
+    "run_startup_checks",
]
diff --git a/backend/config/presets.py b/backend/config/presets.py
index f793227..5d49333 100644
--- a/backend/config/presets.py
+++ b/backend/config/presets.py
@@ -94,6 +94,18 @@ def _cuda_available() -> bool:
    return False

+def probe_cuda_available() -> bool:
+    """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
+
+    Exposed for the per-profile startup checks (``backend/config/startup.py``) so a caller
+    that *wants* the GPU heads-up can pass the result in without the startup module importing
+    torch itself. Best-effort by design: if torch lives only in a detector subprocess env
+    (native WASB), this returns ``False`` in the main process ? which is why the startup checks
+    treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a hard
+    error that could false-fail."""
+    return _cuda_available()
+
def suggest_profile(

```

```

    env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
) -> str:
diff --git a/backend/main.py b/backend/main.py
index 7588d3..8e05ce9 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -29,7 +29,13 @@ from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
    from backend.utils.paths import resolve_under_root, safe_output_filename, validate_input_path
-    from backend.config import load_config as load_app_config, AppConfig, StitchingConfig # new
typed config
+    from backend.config import ( # new typed config
+        load_config as load_app_config,
+        AppConfig,
+        StitchingConfig,
+        enforce_startup_checks,
+        StartupCheckError,
+    )
    from backend.results import Failure # standardized error/result contract
    from backend.reporting import emit_indexer_report
    from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)
@@ -574,6 +580,16 @@ def main():
    global global_config, db_instance
    global_config = load_app_config() # returns typed AppConfig

+    # M5: per-profile startup checks ? fail fast on an incoherent ACTIVE profile (e.g. a
+    # cloud-serving profile pointed at local-only storage) BEFORE any work. A strict no-op when
+    # no deployment.profile is selected (the default), so today's behaviour is unchanged. No
GPU
+    # probe here (the server stays torch-light; the detector healthcheck is the GPU authority).
+    try:
+        enforce_startup_checks(global_config, logger=logger)
+    except StartupCheckError as e:
+        logger.error("[STARTUP] refusing to start: %s", e)
+        sys.exit(1)
+
+    # The CLI --output-dir overrides the configured output directory. It now lives
+    # on the typed model (OutputConfig.output_dir), so every consumer ? including
+    # /api/compile ? reads the same value instead of a write-only runtime hack.
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index fafb35a..c644ede 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -25,7 +25,12 @@ import tempfile
    from typing import Any, List

    from backend import storage
-    from backend.config import load_config
+    from backend.config import (
+        StartupCheckError,
+        enforce_startup_checks,
+        load_config,
+        probe_cuda_available,
+    )
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
@@ -88,9 +93,23 @@ def _write_report_json(video_id: str, segments: List[dict], status: str,
path: str
    }, f, indent=2)

+def _run_startup_checks(config) -> bool:
+    """M5 per-profile startup checks for the worker (the compute process, so it probes CUDA

```

```

+ best-effort). Logs warnings; returns False on a FATAL coherence error so the caller aborts
+ cleanly (rc 2) rather than run an expensive job under an incoherent profile. A no-op
+ (returns
+ True) when no deployment.profile is selected."""
+ try:
+     enforce_startup_checks(config, cuda_available=probe_cuda_available(), logger=logger)
+     return True
+ except StartupCheckError:
+     return False # already logged at ERROR by enforce_startup_checks
+
+
+ def cmd_infer(args: argparse.Namespace) -> int:
+     config = load_config(args.config) if args.config else load_config()
+     _apply_overrides(config, args.set)
+     if not _run_startup_checks(config):
+         return 2
+     storage_cfg = config.storage.model_dump()

+     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
@@ -260,6 +279,8 @@ def cmd_train(args: argparse.Namespace) -> int:

+     config = load_config(args.config) if args.config else load_config()
+     _apply_overrides(config, args.set)
+     if not _run_startup_checks(config):
+         return 2
+     storage_cfg = config.storage.model_dump()

+     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")

```

## 7. user (2026-06-21T10:18:36.087Z)

```

1     """Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
2
3     A deployment profile (`truly-local` / `cloud-serving` / `cpu-mock`) implies a
coherent
4     runtime. These checks fail/warn fast at process startup ? a loud signal before any
5     (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7     * cloud-serving with a non-cloud storage backend ? a hard error (the one
config bug we
8     can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9     bucket can never serve). This is the README §4.3 "loud error before any job".
10    * a GPU mismatch (e.g. `device='cuda'` on a box where CUDA isn't visible, or
`device='auto'`
11    falling back to CPU) ? a warning, never an error: the probe is best-effort
(torch may live
12    only in the detector subprocess env ? see `presets.probe_cuda_available`), and the
detector's
13    own healthcheck is the real authority. A warning surfaces it early without false-
failing.
14    * creds that can't be confirmed (no `GOOGLE_APPLICATION_CREDENTIALS` / not on
Cloud Run) ?
15    a warning: Application Default Credentials can come from the metadata server /
gcloud, which
16    we can't probe offline, so the storage backend's own init is the authority; we just
flag it.
17
18    DEFAULT-OFF: with no profile selected (`profile == ""`) this is a strict no-op
? exactly
19    the pre-M5 behaviour. Only an explicitly-selected profile triggers any check (mirrors
the loader,
20    which applies a preset only for an explicit profile). So nothing changes for today's
manual-knob
21    deployments.

```

```

22     """
23
24     from __future__ import annotations
25
26     import logging
27     import os
28     from dataclasses import dataclass
29     from typing import Any, Mapping, Optional
30
31     LEVEL_ERROR = "error"
32     LEVEL_WARN = "warn"
33
34     # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
35     _CLOUD_STORAGE = ("gcs", "s3")
36     # Env signals that imply Google ADC *might* resolve without an explicit creds file.
37     _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST",
38                       "CLOUDSDK_CONFIG", "GOOGLE_CLOUD_PROJECT")
39
40
41     @dataclass(frozen=True)
42     class StartupCheck:
43         """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
44 up)."""
45         level: str
46         code: str          # stable, greppable code (e.g. "STORAGE_INCOHERENT")
47         message: str
48
49
50     class StartupCheckError(RuntimeError):
51         """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL
52 check."""
53
54         def __init__(self, failures: list[StartupCheck]):
55             self.failures = failures
56             super().__init__(
57                 "deployment profile startup check failed: "
58                 + "; ".join(f"[{c.code}] {c.message}" for c in failures)
59             )
60
61     def _creds_discoverable(env: Mapping[str, str]) -> bool:
62         """True if *some* Google ADC source is hinted by the environment. Best-effort ? a
63 False here
64         only downgrades to a WARNING, never a hard failure (the storage init is the
65 authority)."""
66         return any(env.get(k) for k in _GCP_ENV_HINTS)
67
68     def run_startup_checks(
69         config: Any,
70         *,
71         cuda_available: Optional[bool] = None,
72         env: Optional[Mapping[str, str]] = None,
73     ) -> list[StartupCheck]:
74         """Return the per-profile findings for ``config`` (empty when no profile is
75 selected).
76
77         ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ?
78 GPU checks
79         are skipped; the API server passes ``None``, the worker passes
80         ``presets.probe_cuda_available()``).
81         Pure: no IO, no torch import ? the caller owns the probe so this stays importable
82         everywhere.
83         """

```

```

79     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
80     if not profile:
81         return [] # DEFAULT-OFF: no profile ? no checks (identical to pre-M5).
82
83     e = os.environ if env is None else env
84     storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
85     indexing = getattr(config, "indexing", None)
86     detector_impl = getattr(indexing, "detector_impl", "auto")
87     device = getattr(getattr(indexing, "wasb", None), "device", "cuda")
88
89     checks: list[StartupCheck] = []
90
91     if profile == "truly-local":
92         # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
backend here is
93         # an inconsistent (but not fatal) choice worth surfacing.
94         if storage_backend not in ("local", "gdrive"):
95             checks.append(StartupCheck(
96                 LEVEL_WARN, "STORAGE_UNUSUAL",
97                 f"profile 'truly-local' usually uses local/gdrive storage, but
storage_backend="
98                 f"'{storage_backend}'. Reads will go to a remote backend.",
99             ))
100         # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
101         # best-effort and may be False even when a subprocess detector env has CUDA).
102         if cuda_available is False:
103             if device == "cuda":
104                 checks.append(StartupCheck(
105                     LEVEL_WARN, "CUDA_NOT_VISIBLE",
106                     "device='cuda' but no CUDA GPU is visible to this process. If the
box truly "
107                     "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
108                     "'auto' for the CPU fallback (M4).",
109                 ))
110             elif device == "auto":
111                 checks.append(StartupCheck(
112                     LEVEL_WARN, "CPU_FALLBACK",
113                     "device='auto' resolved to CPU (no CUDA GPU visible). Inference will
run on "
114                     "CPU ? correct, but much slower than a GPU.",
115                 ))
116
117     elif profile == "cloud-serving":
118         # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
119         if storage_backend not in _CLOUD_STORAGE:
120             checks.append(StartupCheck(
121                 LEVEL_ERROR, "STORAGE_INCOHERENT",
122                 f"profile 'cloud-serving' requires a cloud storage backend "
123                 f"'{'/'.join(_CLOUD_STORAGE)}', but storage_backend='{storage_backend}'.
"
124                 "A cloud job cannot read/write a local-only backend.",
125             ))
126         elif storage_backend == "gcs" and not _creds_discoverable(e):
127             checks.append(StartupCheck(
128                 LEVEL_WARN, "CREDS_UNCONFIRMED",
129                 "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on
Application "
130                 "Default Credentials. The GCS backend will fail loudly at init if ADC
can't resolve.",
131             ))
132         if device == "cpu":
133             checks.append(StartupCheck(

```

```

134             LEVEL_WARN, "CLOUD_CPU_DEVICE",
135             "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
136             "detection will not use the GPU.",
137         ))
138     elif cuda_available is False and device in ("cuda", "auto"):
139         checks.append(StartupCheck(
140             LEVEL_WARN, "CLOUD_CUDA_NOT_VISIBLE",
141             "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
142         ))
143
144     elif profile == "cpu-mock":
145         # The mock profile is for CI/regression ? a real detector here defeats the
purpose.
146         if detector_impl != "stub":
147             checks.append(StartupCheck(
148                 LEVEL_WARN, "MOCK_NONSTUB_DETECTOR",
149                 f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
150                 f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
151             ))
152
153     return checks
154
155
156 def enforce_startup_checks(
157     config: Any,
158     *,
159     cuda_available: Optional[bool] = None,
160     env: Optional[Mapping[str, str]] = None,
161     raise_on_error: bool = True,
162     logger: Optional[logging.Logger] = None,
163 ) -> list[StartupCheck]:
164     """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
165     ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.
166
167     Returns the full findings list (so a caller can inspect/surface them). A no-op with
no logging
168     and no raise when no profile is selected (``run_startup_checks`` returns ``[]``)."""
169     log = logger or logging.getLogger(__name__)
170     checks = run_startup_checks(config, cuda_available=cuda_available, env=env)
171     for c in checks:
172         if c.level == LEVEL_ERROR:
173             log.error("[STARTUP] %s: %s", c.code, c.message)
174         else:
175             log.warning("[STARTUP] %s: %s", c.code, c.message)
176     errors = [c for c in checks if c.level == LEVEL_ERROR]
177     if errors and raise_on_error:
178         raise StartupCheckError(errors)
179     return checks
180

```

## 8. assistant (2026-06-21T10:18:39.435Z)

Now let me read the test files and look at the surrounding context in main.py and the presets/config conventions.

## 9. user (2026-06-21T10:18:40.741Z)

```

1     """Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
2
3     L1 unit coverage of ``backend/config/startup.py``: the fail/warn-fast per-profile
checks.
4     Pure ? no torch/GPU/network (the CUDA probe + env are injected). The cardinal invariant:

```

```

5     **no profile selected ? ZERO checks** (default-OFF, identical to pre-M5).
6     """
7     import logging
8
9     import pytest
10
11     from backend.config import AppConfig
12     from backend.config.startup import (
13         LEVEL_ERROR,
14         LEVEL_WARN,
15         StartupCheckError,
16         enforce_startup_checks,
17         run_startup_checks,
18     )
19
20
21     def _cfg(profile="", backend="local", device="cuda", detector_impl="auto"):
22         """Build an AppConfig with arbitrary profile/storage/device combos ? including ones
a
23         preset would never produce (e.g. cloud-serving + local storage), to exercise the
coherence
24         checks. model_validate fills every other field with its default."""
25         return AppConfig.model_validate({
26             "deployment": {"profile": profile},
27             "storage": {"backend": backend},
28             "indexing": {"detector_impl": detector_impl, "wasb": {"device": device}},
29         })
30
31
32     def _codes(checks):
33         return {c.code for c in checks}
34
35
36     # ----- default-OFF
37
38     @pytest.mark.parametrize("backend,device,detector", [
39         ("local", "cuda", "auto"), ("gcs", "cpu", "stub"), ("s3", "auto", "native"),
40     ])
41     def test_no_profile_is_strict_noop(backend, device, detector):
42         """profile='' ? ZERO checks regardless of any other field (the pre-M5 behaviour)."""
43         cfg = _cfg(profile="", backend=backend, device=device, detector_impl=detector)
44         assert run_startup_checks(cfg, cuda_available=False, env={}) == []
45
46
47     def test_enforce_no_profile_never_raises_and_returns_empty():
48         cfg = _cfg(profile="", backend="local")
49         assert enforce_startup_checks(cfg, cuda_available=False, env={}) == []
50
51
52     # ----- truly-local
53
54     def test_truly_local_local_storage_gpu_present_is_clean():
55         cfg = _cfg(profile="truly-local", backend="local", device="cuda")
56         assert run_startup_checks(cfg, cuda_available=True, env={}) == []
57
58
59     def test_truly_local_cuda_strict_without_gpu_warns():
60         cfg = _cfg(profile="truly-local", backend="local", device="cuda")
61         checks = run_startup_checks(cfg, cuda_available=False, env={})
62         assert _codes(checks) == {"CUDA_NOT_VISIBLE"}
63         assert all(c.level == LEVEL_WARN for c in checks) # never fatal ? healthcheck is
the authority
64
65
66     def test_truly_local_auto_without_gpu_warns_cpu_fallback():

```



```

67     cfg = _cfg(profile="truly-local", backend="local", device="auto")
68     checks = run_startup_checks(cfg, cuda_available=False, env={})
69     assert _codes(checks) == {"CPU_FALLBACK"}
70
71
72 def test_truly_local_unprobed_cuda_skips_gpu_checks():
73     """cuda_available=None (not probed, e.g. the API server) ? no GPU heads-up at
all."""
74     cfg = _cfg(profile="truly-local", backend="local", device="cuda")
75     assert run_startup_checks(cfg, cuda_available=None, env={}) == []
76
77
78 def test_truly_local_cloud_storage_warns_unusual():
79     cfg = _cfg(profile="truly-local", backend="gcs", device="auto")
80     checks = run_startup_checks(cfg, cuda_available=True, env={})
81     assert "STORAGE_UNUSUAL" in _codes(checks)
82
83
84 def test_truly_local_gdrive_storage_is_fine():
85     cfg = _cfg(profile="truly-local", backend="gdrive", device="cuda")
86     assert run_startup_checks(cfg, cuda_available=True, env={}) == []
87
88
89 # ----- cloud-serving
90
91 def test_cloud_serving_local_storage_is_fatal():
92     cfg = _cfg(profile="cloud-serving", backend="local")
93     checks = run_startup_checks(cfg, cuda_available=True,
env={"GOOGLE_APPLICATION_CREDENTIALS": "/k"})
94     incoherent = [c for c in checks if c.code == "STORAGE_INCOHERENT"]
95     assert incoherent and incoherent[0].level == LEVEL_ERROR
96
97
98 def test_cloud_serving_gcs_without_creds_warns():
99     cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
100    checks = run_startup_checks(cfg, cuda_available=True, env={})
101    assert _codes(checks) == {"CREDS_UNCONFIRMED"}
102    assert all(c.level == LEVEL_WARN for c in checks)
103
104
105 def test_cloud_serving_gcs_with_creds_is_clean():
106     cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
107     assert run_startup_checks(cfg, cuda_available=True,
env={"GOOGLE_APPLICATION_CREDENTIALS": "/k"}) == []
108
109
110
111 def test_cloud_serving_on_cloud_run_env_skips_creds_warn():
112     """K_SERVICE (Cloud Run) implies ADC via the metadata server ? no creds warning."""
113     cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
114     assert run_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "rally"}) ==
[]
115
116
117 def test_cloud_serving_cpu_device_warns():
118     cfg = _cfg(profile="cloud-serving", backend="gcs", device="cpu")
119     checks = run_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "rally"})
120     assert "CLOUD_CPU_DEVICE" in _codes(checks)
121
122
123 def test_cloud_serving_no_gpu_warns():
124     cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
125     checks = run_startup_checks(cfg, cuda_available=False, env={"K_SERVICE": "rally"})
126     assert "CLOUD_CUDA_NOT_VISIBLE" in _codes(checks)
127
128

```

```

129 # ----- cpu-mock
130
131 def test_cpu_mock_stub_is_clean():
132     cfg = _cfg(profile="cpu-mock", backend="local", detector_impl="stub")
133     assert run_startup_checks(cfg, cuda_available=False, env={}) == []
134
135
136 def test_cpu_mock_real_detector_warns():
137     cfg = _cfg(profile="cpu-mock", backend="local", detector_impl="native")
138     checks = run_startup_checks(cfg, cuda_available=False, env={})
139     assert "MOCK_NONSTUB_DETECTOR" in _codes(checks)
140
141
142 # ----- enforce
143
144 def test_enforce_raises_on_fatal_and_carries_failures(caplog):
145     cfg = _cfg(profile="cloud-serving", backend="local")
146     with caplog.at_level(logging.ERROR):
147         with pytest.raises(StartupCheckError) as ei:
148             enforce_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "x"})
149     assert [c.code for c in ei.value.failures] == ["STORAGE_INCOHERENT"]
150     assert "STORAGE_INCOHERENT" in caplog.text # logged loudly at ERROR before raising
151
152
153 def test_enforce_does_not_raise_on_warnings_only():
154     cfg = _cfg(profile="truly-local", backend="local", device="auto")
155     checks = enforce_startup_checks(cfg, cuda_available=False, env={}) # CPU_FALLBACK
warning
156     assert checks and all(c.level == LEVEL_WARN for c in checks) # returned, not raised
157
158
159 def test_enforce_raise_on_error_false_returns_errors_without_raising():
160     cfg = _cfg(profile="cloud-serving", backend="local")
161     checks = enforce_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "x"},
162                                     raise_on_error=False)
163     assert any(c.code == "STORAGE_INCOHERENT" and c.level == LEVEL_ERROR for c in
checks)
164
165
166 # ----- worker wiring (fail fast before
any work)
167
168 def test_worker_cmd_infer_aborts_rc2_on_incoherent_profile(tmp_path):
169     """The worker's M5 hook aborts cleanly (rc 2) BEFORE any fetch/segmentation when the
active
170     profile is fatally incoherent. The cloud-serving preset sets storage=gcs;
config.json overrides
171     it back to local ? STORAGE_INCOHERENT ? fail fast."""
172     from backend.worker import __main__ as wmain
173
174     cfg = tmp_path / "config.json"
175     cfg.write_text('{"deployment": {"profile": "cloud-serving"}, "storage": {"backend":
"local"}}}')
176     video = tmp_path / "in.mp4"
177     video.write_bytes(b"FAKE")
178     rc = wmain.cmd_infer(wmain.build_parser().parse_args([
179         "infer", "--video-uri", str(video), "--out-uri", str(tmp_path / "out"),
180         "--config", str(cfg), "--scratch", str(tmp_path / "s"), "--segmenter",
"wasb_hybrid"]))
181     assert rc == 2
182

```

## 10. user (2026-06-21T10:18:40.796Z)

```

1 """Profile-parity ? the load-bearing decoupling proof (COMPUTE_DECOUPLED_SERVING M5,

```

```

L4).
2
3     The decoupling guarantee: the same `(trajectory + config)` produces the **same candidate
4     windows + the same segment ranges** regardless of the active deployment profile /
orchestration.
5     Detection is replaced by the deterministic CPU **stub** (no GPU/WSL/network), so ANY
divergence
6     is necessarily a profile/seam leak into the pure core ? exactly what D1 forbids.
7
8     Three arms:
9     * **L4a** ? the WINDOW stage (`trajectory_to_action_windows`) is a pure, profile-
agnostic
10         staticmethod: deterministic across calls + its signature carries no
profile/compute/storage
11         parameter (so it *structurally* cannot branch on a profile).
12     * **L4b** ? the real stub-backed segmenter produces byte-identical segment ranges
under
13         `profile=truly-local` vs `profile=cpu-mock` vs no profile (the only varied knob is
the
14         compute target, which the core never reads).
15     * **L4c** ? the worker (`cmd_infer`, the cloud-serving orchestration) writes an
`intervals.csv`
16         whose ranges match the in-process truly-local run for the same stub trajectory.
17     ""
18     import inspect
19     import json
20
21     from backend.config import load_config
22     from backend.infrastructure.database import Database
23     from backend.pipeline.detectors.stub_runner import StubDetectorRunner
24     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
25
26
27     # A fixed duration so process_video's early ffprobe check passes for the fake
(undecodable) video;
28     # applied identically to every arm so it can never be the source of a divergence.
29     _DURATION = 30.0
30
31
32     def _config(tmp_path, profile):
33         """A config.json applying `profile` (or none), forced to the deterministic CPU stub
+
34         fully-local (no AI handoff) + a tmp output dir. (The stub uses its default synth
params ?
35         `indexing.stub` is not a typed knob, so it deterministically self-configures.)"""
36         data = {
37             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
38             "output": {"output_dir": str(tmp_path / "out")},
39         }
40         if profile:
41             data["deployment"] = {"profile": profile}
42         cfg_file = tmp_path / f"config_{profile or 'none'}.json"
43         cfg_file.write_text(json.dumps(data))
44         return load_config(cfg_file)
45
46
47     def _ranges(segments):
48         """Normalise a segment list to comparable (start, end) tuples (rounded to drop float
noise)."""
49         out = []
50         for s in segments:
51             start = s.get("start_time", s.get("start"))
52             end = s.get("end_time", s.get("end"))
53             out.append((round(float(start), 3), round(float(end), 3)))
54         return out

```

```

55
56
57     def _inprocess_ranges(tmp_path, profile, monkeypatch):
58         tmp_path.mkdir(parents=True, exist_ok=True)
59         # The fake video is undecodable; pin the duration probe so the early validity check
passes.
60         # Applied to the shared module fn ? identical for every profile arm (never a
divergence source).
61
monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
62                     lambda p: _DURATION)
63         db = Database(str(tmp_path / f"db_{profile or 'none'}.sqlite"))
64         cfg = _config(tmp_path, profile)
65         video = tmp_path / "in.mp4"
66         video.write_bytes(b"FAKEVIDEO-PARITY")
67         db.add_video("vidParity", str(video), "processed", []) # FK target for
candidate/play rows
68         seg = WasbHybridSegmenter(db, cfg)
69         result = seg.process_video("vidParity", str(video))
70         return _ranges(result)
71
72
73     # ----- L4a (pure WINDOW
stage)
74
75     def test_window_stage_has_no_profile_parameter():
76         """Structural guard: the WINDOW stage (`trajectory_to_action_windows`) carries no
77         profile/compute/storage parameter, so it *cannot* branch on a deployment profile ?
the
78         precise leak D1 forbids. (Determinism across profiles is exercised end-to-end in
L4b.)"""
79         sig = inspect.signature(StubDetectorRunner.trajectory_to_action_windows)
80         banned = {"profile", "compute_target", "deployment", "storage", "backend"}
81         assert not (banned & set(sig.parameters)), \
82             f"WINDOW stage must not take a profile/compute/storage param; got
{list(sig.parameters)}"
83
84
85     # ----- L4b (in-process
parity)
86
87     def test_inprocess_segment_ranges_identical_across_profiles(tmp_path, monkeypatch):
88         """Same stub trajectory + config under different profiles ? byte-identical segment
ranges.
89         The profiles differ only in compute_target/label, which the pure core never
reads."""
90         local = _inprocess_ranges(tmp_path / "a", "truly-local", monkeypatch)
91         mock = _inprocess_ranges(tmp_path / "b", "cpu-mock", monkeypatch)
92         none = _inprocess_ranges(tmp_path / "c", "", monkeypatch)
93         assert local == mock == none
94         assert len(local) >= 1 # the stub actually produced rallies (the test isn't
vacuous)
95
96
97     # ----- L4c (worker
orchestration)
98
99     def test_worker_intervals_match_inprocess(tmp_path, monkeypatch):
100         """The worker (cmd_infer ? the cloud-serving orchestration) serialises an
intervals.csv whose
101         ranges equal the in-process truly-local run for the same stub trajectory. Proves the
102         fetch/store/serialise orchestration doesn't perturb the core."""
103         from backend.worker import __main__ as wmain
104
105         # In-process reference ranges (this also pins get_video_duration for the worker run

```

```

below).
106     ref = _inprocess_ranges(tmp_path / "ref", "truly-local", monkeypatch)
107
108     # Worker path: stub + skip-AI, validators mocked (fake bytes aren't a real video).
109     class _OK:
110         passed = True
111         validator_name = "fake"
112         message = "ok"
113
114         def model_dump(self):
115             return {"validator_name": "fake", "passed": True, "message": "ok"}
116
117     monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
118                         lambda path, cfg: ([_OK()], {}))
119     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
120
121     cfg_file = tmp_path / "wcfg.json"
122     cfg_file.write_text(json.dumps({
123         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
124         "output": {"output_dir": str(tmp_path / "wout")},
125     }))
126     video = tmp_path / "w.mp4"
127     video.write_bytes(b"FAKEVIDEO-PARITY")
128     out = tmp_path / "bucket"
129
130     rc = wmain.cmd_infer(wmain.build_parser().parse_args([
131         "infer", "--video-uri", str(video), "--out-uri", str(out), "--config",
132         str(cfg_file),
133         "--scratch", str(tmp_path / "scratch"), "--segmenter", "wasb_hybrid"]))
134     assert rc == 0
135
136     # Find the intervals.csv (outputs/<video_id>/intervals.csv).
137     intervals = next(out.rglob("intervals.csv"))
138     rows = intervals.read_text().splitlines()[1:] # drop header
139     worker = [(round(float(r.split(",")[0]), 3), round(float(r.split(",")[1]), 3)) for r
140 in rows]
141     assert worker == ref
142

```

## 11. assistant (2026-06-21T10:18:44.292Z)

Now let me check the surrounding context in main.py (logger setup, import block) and the DeploymentConfig / presets style for M1-M4 consistency. Let me also check how logging is configured in main.py and worker.

## 12. user (2026-06-21T10:18:45.741Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
13    for direct user-facing messages).
14    logging.basicConfig(
15        level=logging.INFO,
16        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
17        datefmt='%H:%M:%S'
18    )

```

```

18     logger = logging.getLogger(__name__)
19     from fastapi import FastAPI, HTTPException
20     from fastapi.middleware.cors import CORSMiddleware
21     from pydantic import BaseModel
22     from typing import List, Dict, Any, Optional
23
24     from fastapi.staticfiles import StaticFiles
25     from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26     from backend.pipeline.validators.base import registry
27     from backend.infrastructure.database import Database
28     from backend.pipeline.segmenters.base import segmenter_registry
29     from backend.pipeline.stitcher.compiler import VideoCompiler
30     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32     from backend.config import ( # new typed config
33         load_config as load_app_config,
34         AppConfig,
35         StitchingConfig,
36         enforce_startup_checks,
37         StartupCheckError,
38     )
39     from backend.results import Failure # standardized error/result contract
40     from backend.reporting import emit_indexer_report
41     from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
42
43     app = FastAPI(title="Sports Highlight Indexer API")
44
45     # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
46     # rejected by browsers per the fetch spec and is an unsafe default to carry into the
47     # auth/tenancy work; this tool uses no cookies, so credentials stay off.
48     app.add_middleware(
49         CORSMiddleware,
50         allow_origins=["*"],
51         allow_credentials=False,
52         allow_methods=["*"],
53         allow_headers=["*"],
54     )
55
56     # Serves static frontend files directly (now under api/ per approved structure)
57     os.makedirs("backend/api/static", exist_ok=True)
58     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
59
60     @app.get("/", response_class=HTMLResponse)

```

### 13. user (2026-06-21T10:18:46.975Z)

```

backend/worker/__main__.py:41:logger = logging.getLogger("backend.worker")
backend/worker/__main__.py:431:    # force=True: basicConfig is a no-op if the root already has
handlers (e.g. a library
backend/worker/__main__.py:434:    logging.basicConfig(
backend/main.py:13:logging.basicConfig(
backend/main.py:18:logger = logging.getLogger(__name__)

```

### 14. assistant (2026-06-21T10:18:50.174Z)

Let me check the worker's logging setup (line 431) and where `\_run\_startup\_checks` is called relative to it, plus the DeploymentConfig Literal and presets style.

### 15. user (2026-06-21T10:18:51.357Z)

```

420     pt.add_argument("--set", action="append", default=[], metavar="k=v")

```

```

421         pt.set_defaults(func=cmd_train)
422         return p
423
424
425     def _setup_logging(verbose: bool) -> None:
426         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
427         diagnostics are
428         VISIBLE ? the worker previously ran with no logging config, so every
429         segmenter/detector log
430         (detector choice, 'Running native WASB inference', availability failures) was
431         silently dropped.
432         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
433         (the CLI
434         entry), not at import, so importing the module never reconfigures a caller's
435         logging."""
436         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
437         library
438         # configured logging at import), which would leave the level at WARNING and re-
439         suppress the
440         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
441         verbosity.
442         logging.basicConfig(
443             level=logging.DEBUG if verbose else logging.INFO,
444             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
445             datefmt="%H:%M:%S",
446             force=True,
447         )
448
449     def main(argv=None) -> int:
450         args = build_parser().parse_args(argv)
451         _setup_logging(getattr(args, "verbose", False))
452         return args.func(args)
453
454     if __name__ == "__main__":
455         sys.exit(main())
456
457

```

## 16. user (2026-06-21T10:18:52.778Z)

```

11:from typing import Any, List, Literal, Optional
15:Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
36:     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
63:     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
105:     substrate: Literal["wasb", "tracknet"] = "wasb"
123:     net_axis: Literal["x", "y"] = "y"
252:     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
304:     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
315:     input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
334:class DeploymentConfig(BaseModel):
335:     """Deployment profile ? one switch that selects a coherent bundle of compute + storage
336:     **Pure / additive:** the empty default profile (````) applies NO preset, so the
337:     pipeline
338:     behaves exactly as before this section existed. A profile is opt-in (``config.json``
339:     ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
340:     these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
341:     knobs
342:     # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
343:     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
344:     # Where the core runs. Set by the profile preset; overridable in config.json. Inert in
345:     M1
351:     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
---PRESETS HEAD---

```

```
"""Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING M1).
```

A *deployment profile* is a single switch that applies a coherent BUNDLE of existing config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one knob at a time (the design in ``docs/COMPUTE\_DECOUPLED\_SERVING/README.md`` §4.3).

Precedence (lowest ? highest), realised by the loader::

```
built-in model defaults < profile preset < config.json < env < worker --set
```

This module is *pure data + pure functions* ? no IO, and (critically) *no torch import at module load*. The loader applies a preset ONLY when a profile is EXPLICITLY selected (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT\_PROFILE`` env var); with no profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` == today). Auto-detect only ever *suggests* a profile ? it never auto-applies one (D15: cloud spend is never entered silently).

```
from __future__ import annotations
```

```
import copy
```

```
import os
```

```
from typing import Any, Mapping, Optional
```

**Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal`` these and ``tests/test\_config\_deployment.py`` pins parity so the two can never drift.**

```
PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
```

```
COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
```

**Each profile maps 1:1 to its canonical compute\_target. Used to apply the preset and to**

**17. assistant (2026-06-21T10:18:58.321Z)**

Now I have good context. Let me verify a few specific concerns:

1. The worker's ``\_run\_startup\_checks`` calls ``probe\_cuda\_available()`` unconditionally even when ``profile=""`` ? need to confirm whether the probe (torch import) runs before the early-return.
2. Circular import risk in ``backend.config.\_\_init\_\_``.
3. Whether ``\_run\_startup\_checks`` runs before ``\_setup\_logging`` is established (it's in ``main()`` ? ``\_setup\_logging`` first, then ``args.func`` ? cmd\_infer, so logging is set up).

Let me trace the probe ordering and check for the unconditional torch import concern, plus run the relevant tests and lint.

**18. user (2026-06-21T10:18:59.437Z)**

**Each profile maps 1:1 to its canonical compute\_target. Used to apply the preset and to when config.json explicitly overrides compute\_target into a state inconsistent with the**

```
COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
    "truly-local": "local_gpu",
    "cloud-serving": "cloud_run",
    "cpu-mock": "cpu_mock",
}
```

**A preset is a partial, nested override dict over the model defaults. It touches ONLY knobs that EXIST today ? M1 is pure/additive: the new ``deployment.{profile,compute\_target}``, ``indexing.{detector\_impl,skip\_ai\_handoff}`` and ``storage.backend``. Later milestones these bundles ? ``input\_backend`` (M2) and the configurable output/workspace base (**



## Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.

```
PROFILE_PRESETS: dict[str, dict[str, Any]] = {
    "truly-local": {
        "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
        # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU box).
        # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
        "indexing": {"skip_ai_handoff": True},
        "storage": {"backend": "local"},
    },
    "cloud-serving": {
        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
        # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor (D11).
        "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
        "storage": {"backend": "gcs"},
    },
    "cpu-mock": {
        "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
        # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression profile.
        "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
        "storage": {"backend": "local"},
    },
}
```

```
def is_known_profile(profile: str) -> bool:
    """True for a non-empty, recognised profile name."""
    return bool(profile) and profile in PROFILE_PRESETS
```

```
def preset_for(profile: str) -> dict[str, Any]:
    """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate the
    module-level table), or ``{}`` for ``""`` / an unknown profile."""
    return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
```

```
def _truthy(val: Optional[str]) -> bool:
    """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
    if val is None:
        return False
    return val.strip().lower() in {"1", "true", "yes", "on"}
```

```
def _cuda_available() -> bool:
    """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here and
    ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no GPU'."""
    try:
        # Intentionally lazy + local: keeps the config-load path torch-free (torch is heavy
        # and optional). Only reached when a caller opts into probe_cuda.
        import torch

        return bool(torch.cuda.is_available())
    except Exception:
        return False
```

### 19. user (2026-06-21T10:19:00.295Z)

```
4:config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
33:    "truly-local": "local_gpu",
34:    "cloud-serving": "cloud_run",
35:    "cpu-mock": "cpu_mock",
44:    "truly-local": {
45:        "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
51:    "cloud-serving": {
```

```

52:         "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
57:     "cpu-mock": {
58:         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
68:     return bool(profile) and profile in PROFILE_PRESETS
116:     - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
117:     - ``CI`` truthy                            ? ``"cpu-mock"``
118:     - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
122:     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
127:         return "cloud-serving"
129:         return "cpu-mock"
131:         return "truly-local"

```

## 20. assistant (2026-06-21T10:19:05.885Z)

Now I've confirmed a real concern: in the worker's `\_run\_startup\_checks`, `probe\_cuda\_available()` is called eagerly as an argument to `enforce\_startup\_checks`, so the torch import probe runs **before** `\_run\_startup\_checks` does its `profile==""` early-return. This means a default-OFF worker invocation (no profile) pays the torch import cost. Let me verify this is actually a behavior change vs pre-M5 and whether it's the "best-effort, lazy torch import" the design promised, and whether it affects the default-OFF byte-identical contract.

Let me trace precisely and confirm the probe is unconditional in the worker path.

## 21. user (2026-06-21T10:19:07.195Z)

```

96     def _run_startup_checks(config) -> bool:
97         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
98         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
99         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A no-op
(return
100         True) when no deployment.profile is selected."""
101         try:
102             enforce_startup_checks(config, cuda_available=probe_cuda_available(),
logger=logger)
103             return True
104         except StartupCheckError:
105             return False # already logged at ERROR by enforce_startup_checks
106
107
108     def cmd_infer(args: argparse.Namespace) -> int:
109         config = load_config(args.config) if args.config else load_config()
110         _apply_overrides(config, args.set)
111         if not _run_startup_checks(config):
112             return 2
113         storage_cfg = config.storage.model_dump()
114
115         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")

```

## 22. assistant (2026-06-21T10:19:12.243Z)

Confirmed. `probe\_cuda\_available()` is evaluated eagerly in the `enforce\_startup\_checks(...)` argument list, so the torch import probe runs on **every** worker `cmd\_infer`/`cmd\_train` call including when `profile==""` (default-OFF). The `\_run\_startup\_checks` early-return on empty profile happens **after** the probe already ran. This is a real default-OFF deviation (the probe has a side effect: importing torch, which can be heavy/slow, and `presets.cuda\_available()` even runs `torch.cuda.is\_available()`).

Let me now run the new tests, ruff, and mypy to catch any CI-lane issues, and check whether there's an existing pattern the worker had before (was the worker already importing torch elsewhere on this path?).

### 23. user (2026-06-21T10:19:23.673Z)

..... [100%]  
25 passed in 7.75s

### 24. user (2026-06-21T10:19:27.240Z)

All checks passed!

### 25. user (2026-06-21T10:19:30.187Z)

Success: no issues found in 2 source files

### 26. assistant (2026-06-21T10:19:32.991Z)

Ruff and mypy are clean. Let me check the repo's CI mypy invocation to match it exactly (strict flags), and verify whether the worker previously imported torch on the infer path (to judge whether the eager probe is a *\*new\** cost or already paid).

### 27. user (2026-06-21T10:19:34.688Z)

```
.github/workflows/ci.yml:16: # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle +  
bugbear.  
.github/workflows/ci.yml:17: # Also the repo-wide syntax guard ? ruff reports invalid-syntax on  
every  
.github/workflows/ci.yml:31:         run: ruff check . --output-format github  
.github/workflows/ci.yml:33: # Type gate (mypy.ini at repo root): lenient green baseline with  
an explicit  
.github/workflows/ci.yml:48:         run: mypy backend training  
.github/workflows/ci.yml:87:         # (pytest, pytest-cov, httpx for the fastapi TestClient,  
ruff, mypy).  
pyproject.toml:4:# (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same  
pyproject.toml:45:     "ruff",  
pyproject.toml:46:     "mypy",
```

### 28. user (2026-06-21T10:19:35.825Z)

```
backend/worker/__main__.py:32:     probe_cuda_available,  
backend/worker/__main__.py:102:         enforce_startup_checks(config,  
cuda_available=probe_cuda_available(), logger=logger)  
backend/pipeline/detectors/device.py:37:     cuda_available: bool        # probed in the runner's  
own torch env (not the main process)  
backend/pipeline/detectors/device.py:44:         return CUDA if self.cuda_available else CPU  
backend/pipeline/detectors/device.py:50:         return self.requested == AUTO and not  
self.cuda_available  
backend/pipeline/detectors/device.py:56:         return self.requested == CUDA and not  
self.cuda_available  
backend/pipeline/detectors/native_wasb_runner.py:56:_CUDA_PROBE_CODE = "import torch;  
print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"  
backend/pipeline/detectors/native_wasb_runner.py:107:         self._cuda_available:  
Optional[bool] = None  
backend/pipeline/detectors/native_wasb_runner.py:116:     def _probe_cuda(self) -> bool:  
backend/pipeline/detectors/native_wasb_runner.py:127:         if self._cuda_available is None:  
backend/pipeline/detectors/native_wasb_runner.py:128:             self._cuda_available =  
self._probe_cuda()  
backend/pipeline/detectors/native_wasb_runner.py:129:         return self._cuda_available  
backend/pipeline/detectors/native_wasb_runner.py:182:         self._cuda_available = "cuda True"  
in out  
backend/pipeline/detectors/native_wasb_runner.py:183:         ctx =  
DeviceContext(self.cfg.device, self._cuda_available)  
backend/pipeline/detectors/person_detector.py:29:import torch  
backend/pipeline/detectors/wasb_infer.py:397:     import torch  
backend/pipeline/detectors/wasb_runner.py:75:         f"python -c \"import torch;  
print('torch', torch.__version__, "
```

### 29. assistant (2026-06-21T10:19:41.003Z)

Key insight: the M4 design (`native\_wasb\_runner.py`) deliberately probes CUDA in a `**subprocess** torch env`, NOT the main process ? and the design docstrings in ``presets.probe_cuda_available`` and ``startup.py`` explicitly say torch "may live only in the detector subprocess env." So in the main worker process, ``import torch`` typically fails and returns False harmlessly. The eager call still *attempts* the import on every invocation though.

Let me verify the magnitude of the default-OFF concern: does ``probe_cuda_available()`` actually short-circuit cheaply, or does it do real work, and confirm the worker path didn't import torch into the main process before M5. Let me also confirm there are no other CI mypy strictness issues by running the exact CI command.

### 30. user (2026-06-21T10:19:44.914Z)

```
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 126 source files
```

### 31. user (2026-06-21T10:19:45.140Z)

EXIT\_PATTERN\_DONE

### 32. assistant (2026-06-21T10:19:48.782Z)

mypy passes on the full ``backend training`` (CI command), and ruff repo-wide is clean for the changed files. Now let me confirm the circular-import question concretely by importing the config package fresh, and check whether ``_run_startup_checks(config)`` lacks a type annotation (mypy lenient baseline allows it, but it's an M1-M4 consistency point ? note `presets/startup` use full annotations).

### 33. user (2026-06-21T10:19:50.791Z)

```
config import OK
all M5 symbols import OK
```

### 34. user (2026-06-21T10:19:51.843Z)

```
15:     a warning: Application Default Credentials can come from the metadata server /
gcloud, which
24:from __future__ import annotations
26:import logging
27:import os
28:from dataclasses import dataclass
29:from typing import Any, Mapping, Optional
77:     Pure: no IO, no torch import ? the caller owns the probe so this stays importable
everywhere.
92:         # truly-local reads bytes from the local FS or a mounted Drive ? a cloud backend
here is
---worker _run_startup_checks signature---
96:def _run_startup_checks(config) -> bool:
```

### 35. assistant (2026-06-21T10:19:59.673Z)

No circular import ? ``backend/config/startup.py`` imports only `stdlib`` (``logging``, ``os``, ``dataclasses``, ``typing``), not back into ``backend.config``. So ``__init__`` importing ``startup`` is safe.

Let me check the worker's logging timing concern (is `\_run\_startup\_checks` guaranteed to run after `\_setup\_logging`?) and confirm there's no double logging-config or import-time emission. The worker logger is `"backend.worker"` and `\_setup\_logging` runs in `main()` before `args.func`. But tests call `cmd\_infer` directly without `\_setup\_logging` ? that's fine (logging just uses defaults). Let me also verify the `[STARTUP]` prefix convention matches existing repo usage.

#### 36. user (2026-06-21T10:20:03.265Z)

```
1 "[API]
1 "[JOBS]
3 "[STARTUP]
```

#### 37. assistant (2026-06-21T10:20:09.595Z)

The `[STARTUP]` bracketed-prefix style matches the existing repo convention (`[API]`, `[JOBS]`). Logging is via the standard `logging` module passed by the caller, with no import-time emission and no extra `basicConfig` in `startup.py`. Good.

Now let me assess the one substantive finding (eager probe on default-OFF worker path) more rigorously ? measure whether it's actually a behavioral cost vs pre-M5. The pre-M5 worker `cmd\_infer` did NOT import torch into the main process at this point (torch import was deferred to the detector subprocess). The eager `probe\_cuda\_available()` attempts `import torch` in-process on every call regardless of profile.

Let me confirm torch isn't already imported in-process by the time `cmd\_infer` runs (i.e., is the probe genuinely adding an import attempt).

#### 38. user (2026-06-21T10:20:22.444Z)

```
torch already imported by worker module load? True
probe result: True took_ms: 44.38
torch in sys.modules after probe? True
```

#### 39. assistant (2026-06-21T10:20:30.433Z)

Important finding: torch is **already imported** by the worker module load (some import chain pulls it in ? e.g. `person\_detector.py` imports torch at module level). So in this environment, the probe doesn't add a fresh torch import; `import torch` hits the module cache. The probe cost here is dominated by `torch.cuda.is\_available()` (~44ms first call, cached after). On a machine without torch, the probe returns False cheaply via the `except`.

So the eager-probe-on-default-OFF concern is **real but minor**: it does run `probe\_cuda\_available()` unconditionally on every worker `cmd\_infer`/`cmd\_train`, including `profile==""`, doing work (a `torch.cuda.is\_available()` call or an import attempt) whose result is then discarded by the early-return. It doesn't break the byte-identical *output* contract (the result is unused when `profile==""`), but it's wasted work and a slight deviation from the "pure/lazy, only when a profile is active" discipline that `presets.cuda\_available` and the M4 subprocess design carefully maintain. The clean fix is to gate the probe behind a profile check, or pass a lazy callable.

Let me verify there are no other consumers/tests asserting `\_run\_startup\_checks` is annotation-typed, and do a final check on the runlog template file (the other diff item) for hygiene.

#### 40. user (2026-06-21T10:20:32.896Z)

```
.../runlogs/RUNLOG_truly-local_TEMPLATE.md          | 49 ++++++
1 file changed, 49 insertions(+)
---
Binary file backend/config/__pycache__/startup.cpython-313.pyc matches
Binary file backend/config/__pycache__/__init__.cpython-313.pyc matches
backend/main.py:36:     enforce_startup_checks,
backend/main.py:588:     enforce_startup_checks(global_config, logger=logger)
backend/worker/__main__.py:30:     enforce_startup_checks,
```

```
backend/worker/__main__.py:96:def _run_startup_checks(config) -> bool:
backend/worker/__main__.py:102:     enforce_startup_checks(config,
cuda_available=probe_cuda_available(), logger=logger)
backend/worker/__main__.py:105:     return False # already logged at ERROR by
enforce_startup_checks
backend/worker/__main__.py:111:     if not _run_startup_checks(config):
backend/worker/__main__.py:282:     if not _run_startup_checks(config):
```

#### 41. assistant (2026-06-21T10:20:35.719Z)

I have a complete picture. Let me do a final consistency check: the `\_\_pycache\_\_` binary files showing in the grep ? confirm they aren't tracked/committed in this diff (would be a hygiene issue).

#### 42. user (2026-06-21T10:20:37.204Z)

```
pycache_committed_check_done
backend/config/__pycache__/startup.cpython-313.pyc
pycache IS gitignored
```

#### 43. assistant (2026-06-21T10:20:43.623Z)

No `\_\_pycache\_\_` committed ? properly gitignored.

I've completed my review. Summary of what I verified:

- New M5 tests pass (25 passed), ruff clean repo-wide, mypy clean on full `backend training` (the CI command).
- No circular import: `startup.py` imports only stdlib; `backend.config` package imports cleanly.
- `[STARTUP]` logging prefix matches repo convention (`[API]`, `[JOBS]`); logger passed by caller, no import-time emission, no extra `basicConfig`.
- DEFAULT-OFF output contract holds (empty profile ? `[ ]`, no logs, no raise).
- One real hygiene/discipline finding: the worker evaluates `probe\_cuda\_available()` eagerly on every `cmd\_infer`/`cmd\_train`, including `profile=""`, doing work that the early-return then discards.

Let me record the findings.

#### 44. user (2026-06-21T10:20:57.822Z)

Structured output provided successfully

#### 45. assistant (2026-06-21T10:21:08.256Z)

Review complete. From the REPO CONVENTIONS / INTEGRATION / HYGIENE lens, the M5 change is in good shape ? I verified the tests pass (25/25), ruff is clean repo-wide, mypy is clean on the full `backend training` CI target, there's no circular import, the `[STARTUP]` logging prefix matches the repo convention with no import-time emission or double `basicConfig`, and the DEFAULT-OFF output contract holds.

Two grounded findings (both non-blocking):

- **minor** ? `backend/worker/\_\_main\_\_.py:102`: `probe\_cuda\_available()` is evaluated eagerly on every `cmd\_infer`/`cmd\_train`, including the `profile=""` default-OFF case, where `run\_startup\_checks` then discards the result via its early-return. Output stays byte-identical, but it re-introduces a main-process torch touch (`torch.cuda.is\_available()`, ~44ms) on the default path ? counter to the lazy/subprocess discipline the M4 runner and `presets.cuda\_available` carefully keep. The server side correctly passes `cuda\_available=None`. Fix: gate the probe behind a profile check (or pass it as a deferred callable).
- **nit** ? `backend/worker/\_\_main\_\_.py:96`: `\_run\_startup\_checks(config)` leaves `config` unannotated while the rest of M5 is fully typed; lenient CI tolerates it. Annotate as `config: Any`.